তোমার দেওয়া `settings.py` হলো একটি **Django project configuration file**। এখানে পুরো Django project কিভাবে কাজ করবে সেটা configure করা থাকে। নিচে **একটা একটা করে সব term সহজভাবে explain করছি।**

1️⃣ Import Section

```
from decouple import config
from pathlib import Path
```

`from decouple import config`

- `.env` file থেকে **secret data load করার জন্য** ব্যবহার হয়
- যেমন:
  - SECRET_KEY
  - DATABASE_PASSWORD
  - API_KEY

Example `.env`

```
SECRET_KEY=django-secret-123456
```

settings.py

```
SECRET_KEY = config('SECRET_KEY')
```

👉 এতে secret key code এর মধ্যে `hardcode` থাকে না।

`from pathlib import Path`

Python এর **file path manage করার modern library**।

Example:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

মানে project root directory।

Example structure

```
project/
    manage.py
    config/
        settings.py
```

`BASE_DIR` = project folder

2️⃣ BASE_DIR

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

👉 Project এর root directory।

Example

```
/home/user/project/
```

এটা ব্যবহার করা হয়:

```
db.sqlite3
static
media
templates
```

path বানানোর জন্য।

Example

```
BASE_DIR / 'db.sqlite3'
```

3️⃣ SECRET_KEY

```
SECRET_KEY = config('SECRET_KEY')
```

👉 Django security key

এটা ব্যবহার হয়:

- password hashing
- session encryption

- CSRF token
- cookies

⚠️ production এ secret রাখতে হয়।

4️⃣ DEBUG

```
DEBUG = True
```

👉 Development mode enable করে।

DEBUG = True হলে

- error page details দেখায়
- debug information দেখায়

Example

```
Traceback
line number
error type
```

⚠️ Production এ

```
DEBUG = False
```

5️⃣ ALLOWED_HOSTS

```
ALLOWED_HOSTS = []
```

👉 কোন domain থেকে request আসতে পারবে।

Example

```
ALLOWED_HOSTS = [
    "localhost",
    "127.0.0.1",
    "mywebsite.com"
]
```

Production এ দরকার।

6️⃣ INSTALLED_APPS

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
```

👉 Django project এ কোন কোন **app enable থাকবে**।

Built-in apps

App	কাজ
admin	admin panel
auth	authentication
contenttypes	model relation
sessions	login session
messages	flash message
staticfiles	static file

Third party apps

```
rest_framework
debug_toolbar
```

`rest_framework`

👉 Django REST API বানানোর library

`debug_toolbar`

👉 request performance debug tool

দেখায়:

```
SQL query
request time
```

```
memory
headers
```

Custom apps

```
users
api
```

👉 তুমি নিজে বানিয়েছ।

Example

```
python manage.py startapp users
```

7️⃣ MIDDLEWARE

```
MIDDLEWARE = [
    'debug_toolbar.middleware.DebugToolbarMiddleware',
```

Middleware = **request/response processing layer**

Flow:

```
Request
 ↓
Middleware
 ↓
View
 ↓
Response
```

Example middleware

SecurityMiddleware

```
django.middleware.security.SecurityMiddleware
```

Security headers manage করে।

SessionMiddleware

```
django.contrib.sessions.middleware.SessionMiddleware
```

User login session handle করে।

CommonMiddleware

```
django.middleware.common.CommonMiddleware
```

URL normalize করে।

Example

```
/about → /about/
```

CSRF Middleware

```
django.middleware.csrf.CsrfViewMiddleware
```

👉 CSRF attack prevent করে।

AuthenticationMiddleware

```
django.contrib.auth.middleware.AuthenticationMiddleware
```

User login status check করে।

Example

```
request.user
```

MessageMiddleware

```
django.contrib.messages.middleware.MessageMiddleware
```

Flash message support।

Example

```
Login successful
Password changed
```

Clickjacking protection

```
django.middleware.clickjacking.XFrameOptionsMiddleware
```

Iframe attack prevent করে।

8️⃣ ROOT_URLCONF

```
ROOT_URLCONF = 'config.urls'
```

👉 Main URL file।

Example

```
config/
    urls.py
```

এখানে সব routes থাকে।

9️⃣ TEMPLATES

```
TEMPLATES = [
```

👉 HTML template configuration।

BACKEND

```
django.template.backends.django.DjangoTemplates
```

Django template engine।

DIRS

```
'DIRS': [],
```

Custom template folder।

Example

```
templates/
```

APP_DIRS

```
True
```

Django apps এর ভিতরের template detect করে।

Example

```
users/templates/
```

---

context_processors

Template এ data inject করে।

Example

```
request
user
messages
```

🔟 WSGI_APPLICATION

```
WSGI_APPLICATION = 'config.wsgi.application'
```

WSGI = Web Server Gateway Interface

Django কে web server এর সাথে connect করে।

Example servers

```
gunicorn
uwsgi
```

1️⃣ DATABASE

```
DATABASES = {
```

Database configuration।

SQLite

```
'ENGINE': 'django.db.backends.sqlite3'
```

Default database।

NAME

```
BASE_DIR / 'db.sqlite3'
```

Database file location।

Production এ usually:

```
PostgreSQL
MySQL
```

1️⃣2️⃣ AUTH_PASSWORD_VALIDATORS

Password security rules।

UserAttributeSimilarityValidator

Password user name এর সাথে similar না হয়।

Example ❌

```
username = atiar
password = atiar123
```

MinimumLengthValidator

Minimum length।

Example

```
8 characters
```

CommonPasswordValidator

Common password block।

Example

```
123456
password
```

NumericPasswordValidator

Pure number password block।

Example

```
12345678
```

1️⃣3️⃣ Internationalization

```
LANGUAGE_CODE = 'en-us'
```

Default language।

TIME_ZONE

```
UTC
```

Time zone।

Bangladesh হলে:

```
Asia/Dhaka
```

USE_I18N

```
True
```

Internationalization enable।

USE_TZ

```
True
```

Timezone support।

1️⃣4️⃣ STATIC FILES

```
STATIC_URL = 'static/'
```

Static file URL।

Example

```
CSS
JS
Images
```

Example

```
/static/css/style.css
```

1️⃣5️⃣ Django REST Framework

```
REST_FRAMEWORK={
```

DRF global settings।

DEFAULT_PERMISSION_CLASSES

```
"rest_framework.permissions.AllowAny"
```

API কে public করে।

মানে:

```
login ছাড়াই access করা যাবে
```

Other options:

```
IsAuthenticated
IsAdminUser
```

1️⃣6️⃣ INTERNAL_IPS

```
INTERNAL_IPS=[
    '127.0.0.1',
]
```

Debug toolbar show করার জন্য।

Local machine IP।

📊 Full Django Architecture (simplified)

```
Browser
   |
   ▼
URL (urls.py)
   |
   ▼
Middleware
   |
   ▼
View
   |
   ▼
Model (Database)
   |
   ▼
Response
```

🚀 যদি চাও আমি তোমাকে এটা ও দেখাতে পারি

1️⃣ Professional Django settings.py structure
2️⃣ Production settings (PostgreSQL + Redis + Celery)
3️⃣ Best Django project architecture (used by big companies)
4️⃣ DRF production ready settings

যা তোমার **AI CCTV + Django backend project** এর জন্য খুব কাজে লাগবে।

তুমি যেহেতু **Django + DRF backend** নিয়ে কাজ করছ এবং ভবিষ্যতে **AI CCTV / ML system** বানাতে চাও (আগের কথোপকথনে বলেছিলে), তাই আমি তোমাকে **industry-level Django setup** দেখাচ্ছি। এটা বড় কোম্পানির architecture-এর কাছাকাছি।

1️⃣ Professional Django `settings.py` Structure

বড় project এ **একটা settings.py না রেখে multiple settings file ব্যবহার করা হয়**।

Project structure

```
backend/
│
├── manage.py
│
├── config/
│   ├── __init__.py
│   │
│   ├── settings/
│   │    ├── __init__.py
│   │    ├── base.py
│   │    ├── dev.py
│   │    ├── prod.py
│   │
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
```

Explanation

File	কাজ
`base.py`	common settings
`dev.py`	development settings
`prod.py`	production settings

---

base.py

```python
from pathlib import Path
from decouple import config

BASE_DIR = Path(__file__).resolve().parent.parent.parent

SECRET_KEY = config("SECRET_KEY")

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.staticfiles",

    "rest_framework",
```

```
    "corsheaders",

    "users",
    "api",
]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
]

STATIC_URL = "/static/"
MEDIA_URL = "/media/"
```

---

dev.py

```
from .base import *

DEBUG = True

ALLOWED_HOSTS = ["*"]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

---

prod.py

```
from .base import *

DEBUG = False

ALLOWED_HOSTS = ["yourdomain.com"]

SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

Run server

```
python manage.py runserver --settings=config.settings.dev
```

Production:

```
python manage.py runserver --settings=config.settings.prod
```

2️⃣ Production Settings (PostgreSQL + Redis + Celery)

Production Django সাধারণত এই stack ব্যবহার করে।

```
Nginx
   ↓
Gunicorn
   ↓
Django
   ↓
PostgreSQL
   ↓
Redis
   ↓
Celery Worker
```

PostgreSQL Database

Install

```
pip install psycopg2-binary
```

settings

```
DATABASES = {
 "default": {
     "ENGINE": "django.db.backends.postgresql",
     "NAME": config("DB_NAME"),
     "USER": config("DB_USER"),
     "PASSWORD": config("DB_PASSWORD"),
     "HOST": config("DB_HOST"),
```

```
    "PORT": 5432,
 }
}
```

Redis (Cache + Celery broker)

Install

```
pip install redis django-redis
```

settings

```
CACHES = {
 "default": {
     "BACKEND": "django_redis.cache.RedisCache",
     "LOCATION": "redis://127.0.0.1:6379/1",
 }
}
```

Celery (Background tasks)

Install

```
pip install celery redis
```

celery.py

```
import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings.prod")

app = Celery("config")

app.config_from_object("django.conf:settings", namespace="CELERY")

app.autodiscover_tasks()
```

---

settings

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
```

---

example task

```
from celery import shared_task

@shared_task
def send_email():
    print("Email sent")
```

Run worker

```
celery -A config worker -l info
```

3️⃣ Best Django Project Architecture (Used by Companies)

Enterprise Django structure সাধারণত এমন হয়।

```
backend/
│
├── manage.py
│
├── config/
│   ├── settings
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── apps/
│   ├── users/
│   ├── accounts/
│   ├── cameras/
│   ├── alerts/
│   └── analytics/
│
├── core/
│   ├── permissions.py
│   ├── pagination.py
```

```
│   ├── utils.py
│   └── mixins.py
│
├── services/
│   ├── email_service.py
│   ├── ai_service.py
│
├── api/
│   ├── v1/
│   │   ├── urls.py
│   │   ├── views.py
│   │   └── serializers.py
│
├── requirements/
│   ├── dev.txt
│   └── prod.txt
```

কেন এটা ভালো

Layer	কাজ
apps	business logic
core	reusable utilities
services	external integrations
api	API layer

4️⃣ DRF Production Ready Settings

Production DRF এ এই settings খুব গুরুত্বপূর্ণ।

Authentication

```
REST_FRAMEWORK = {
 "DEFAULT_AUTHENTICATION_CLASSES": [
     "rest_framework_simplejwt.authentication.JWTAuthentication",
 ],
}
```

Install

```
pip install djangorestframework-simplejwt
```

Permissions

```
REST_FRAMEWORK = {
 "DEFAULT_PERMISSION_CLASSES": [
     "rest_framework.permissions.IsAuthenticated",
 ]
}
```

Pagination

```
REST_FRAMEWORK = {
 "DEFAULT_PAGINATION_CLASS":
 "rest_framework.pagination.PageNumberPagination",

 "PAGE_SIZE": 20
}
```

Throttling (API Rate Limit)

```
REST_FRAMEWORK = {
 "DEFAULT_THROTTLE_CLASSES": [
     "rest_framework.throttling.UserRateThrottle",
 ],
 "DEFAULT_THROTTLE_RATES": {
     "user": "1000/day"
 }
}
```

Filtering

```
pip install django-filter
```

```
REST_FRAMEWORK = {
 "DEFAULT_FILTER_BACKENDS": [
     "django_filters.rest_framework.DjangoFilterBackend"
 ]
}
```

🚀 Real Production Django Stack

Large companies usually use:

```
React / Next.js
       ↓
API Gateway
       ↓
Django REST API
       ↓
PostgreSQL
       ↓
Redis
       ↓
Celery
       ↓
Docker
       ↓
Kubernetes
```

🎯 তোমার জন্য বিশেষভাবে বলছি

তুমি যেহেতু করতে চাও:

- **AI CCTV system**
- **Django backend**
- **ML model integration**

তোমার architecture ideally হওয়া উচিত:

```
React Dashboard
      ↓
Django DRF API
      ↓
Redis Queue
      ↓
```

```
Celery Worker
      ↓
PyTorch Model
      ↓
PostgreSQL
```

✅ চাইলে আমি তোমাকে **একটা ready-to-use GitHub level Django backend architecture** বানিয়ে দিতে পারি যেখানে থাকবে:

- JWT Auth
- Redis
- Celery
- DRF
- PostgreSQL
- Docker
- ML inference API

যেটা **AI surveillance system এর জন্য perfect backend** হবে।